[Experiment, Analysis, and Benchmark] Consolidate or Separate? Usually Neither: When Merging Cloud Data Warehouses Pays

Khadarvali Shaik
Independent Researcher
Richmond, VA, USA
khadarvali.shaik@ieee.org

Syam Sajit Dondapati
American International Group
New York, USA
dsyam22@gmail.com

***Abstract*—Merging cloud data warehouses saves money when their workloads run at the same time, not when they sit idle. Under per-second billing, warehouses that run side by side each pay for the seconds they share, and a merged warehouse pays for them once; idle time, the usual reason to consolidate, is removed more cheaply by a shorter auto-suspend timeout. We derive this from the billing rule, confirm it on Snowflake—three real merges saved 49.8%, 20.1% and 0.0% of metered credits against 49.8%, 20.0% and 0.0% predicted—and test it on two weeks of Snowflake's public fleet trace: 134 groups of warehouses that serve the same database, 388 warehouses and 18.6 million queries, replayed under a billing model checked against Snowflake's own records (running time within 0.6%), with size scaling measured on Snowflake. Under a 1.2× latency bound, auto-suspend alone removes 24% of the billed compute (60% for the median group), assuming every warehouse runs the 600-second default; resizing almost never pays; and consolidation adds nothing for the median group once suspend is tuned. Where it does pay—21 of 134 groups—it cuts the best tuned bill by 5–70% (median 12%); these groups hold 98% of the fleet's consolidation saving, 15% of the tuned total, and one 17-warehouse group holds two thirds of it (without it, 5.5%). Across the 79 same-size merges the search makes, the saving predicted from overlap alone matches the replay's to within 0.2% in total. A rule on the overlap of running periods, computed from query history alone, finds the groups where merging pays with a leave-one-out balanced accuracy of 0.80; fitted on one week of history, it predicts which groups pay the next week with balanced accuracy 0.86–0.88. The findings hold across seven sensitivity settings. All code, results, calibration logs and the pre-registration are public.**

I. Introduction

Most organisations do not run one Snowflake warehouse but dozens, created team by team and workload by workload. Each one is billed for every second it runs and keeps running for a while after its last query [1], [2], so a team that issues a two-second dashboard query every few minutes pays mostly for idle time. Platform teams have three remedies. They can lower the auto-suspend timeout, so that idle warehouses stop sooner [3]. They can resize warehouses, trading a higher rate for shorter runs. And they can consolidate warehouses, so that several teams share one running warehouse instead of each paying for its own idle tails. Vendor guidance discusses the first two; engine-side systems scale resources automatically [4], [5], [6]. None of this answers the question a platform team faces when it looks at its own fleet: *given our warehouses, how much does each remedy save, should we merge some of them, and which?*

This paper is an evaluation of those existing remedies on real workloads. We replay two weeks of Snowflake's public fleet trace, Snowset [7], for every group of warehouses in it that serve the same database, under a billing model we validate against Snowflake's own records, and with the effect of warehouse size measured on Snowflake rather than assumed. For each group we compare the remedies one at a time and a joint search over all of them, under a latency bound. We ask four questions:

- **RQ1. Where do the savings come from?** Auto-suspend comes first. Against the 600 s default, tuning it alone saves 24% of billed compute across 134 groups (median group 60%); the joint search saves 35%. Resizing almost never pays under a latency bound, and the reason follows from how queries scale with size.
- **RQ2. What does consolidation add?** For the median group, nothing. In 21 of 134 groups merging cuts the tuned bill by 5–70%, and those groups hold 98% of the fleet's consolidation saving; one 17-warehouse group alone holds two thirds, and without it consolidation adds 5.5% of the tuned total.
- **RQ3. When does it pay?** We expected lightly used, idle-heavy warehouses to benefit. The data say the opposite: merging pays when members' running periods overlap, because concurrent running time is billed once. We derive this from the billing rule, real merges on Snowflake save the predicted amount to within 0.1 points, and a rule on overlap alone reaches a leave-one-out balanced accuracy of 0.80.
- **RQ4. How robust are these answers?** They hold under seven changes to the latency bound, concurrency, resume delay and size model; greedy search matches exhaustive search in all 42 groups where both can run; and the size model predicts held-out sizes far better than the simple alternatives.

The study was pre-registered: definitions, options, metrics

and sensitivity variants were written down with timestamps before the corresponding runs, and every deviation is reported (Section IV-E). All code, raw results, calibration logs and the plan are public, and one command regenerates every number in this paper (Section X). The paper asks only which warehouses should exist, not how the bill of a shared warehouse is divided among its users.

II. Background: How a Warehouse Is Billed

A Snowflake virtual warehouse of size s consumes credits at a rate proportional to s while it runs (X-Small $= 1$ credit per hour, doubling with each size) [1]. It resumes when a query arrives while it is suspended, keeps running while queries execute and for AUTO_SUSPEND $= a$ seconds after the last one ends, and is billed per second with a 60-second minimum each time it resumes [2]. Other services bill similar shapes: Databricks SQL warehouses accrue charges while idle until an auto-stop timeout [8]; Redshift Serverless bills per second with a 60-second minimum [9]; BigQuery bills autoscaled capacity per second with a one-minute minimum and holds it for at least 60 seconds [10].

For a warehouse whose queries execute on intervals $[s_q, e_q]$, let its *coverage* be $A = \bigcup_q [s_q, e_q + a]$. Each connected component of A is one running period, and the billed time is $\sum_C \max(60, |C|)$ over those components. When $a \geq 60$ every component ends with a full tail of at least 60 seconds, so the minimum never binds and the billed time is simply $|A|$, the length of the coverage.

Proposition 1 (What merging saves): Let warehouses $1, \ldots, k$ of the same size s have coverages $A_1, \ldots, A_k$ at $a \geq 60$, and merge them into one warehouse of size s in which every query keeps its timing. The merged warehouse is billed $s\,|\bigcup_i A_i|$, and the saving is

$$s\Big(\sum_i |A_i| - \big|\bigcup_i A_i\big|\Big) \;=\; s \int \big(n(x) - 1\big)^+ \, dx,$$

where $n(x)$ is the number of warehouses running at time x.

Proof: The merged warehouse's coverage is the union of the members' coverages, since it is the union over all their queries. Its billed time is that union's length because $a \geq 60$. For each x, the separate warehouses bill $n(x)$ seconds and the merged one bills $\mathbf{1}[n(x) \geq 1]$; integrating the difference gives the result. ∎

The saving from a merge is exactly the time during which more than one member would be running—their *overlap*. It does not depend on how idle the members are: two idle warehouses that take turns have $n(x) \leq 1$ everywhere and gain nothing, while two busy warehouses that run side by side gain everything they share. Real merges deviate from the proposition in two ways, both costs: queries from several members may queue for the merged warehouse's concurrency slots, and members of different sizes must move to one rate, usually the largest to keep latency. Figure 1 illustrates the two extremes. The proposition makes a prediction the rest of the paper tests: on the real fleet, the saving of every merge should equal its members' overlap, net of the rate change (Figure 9, Section VI-C), and a real merge on Snowflake should save it (Section V-C).

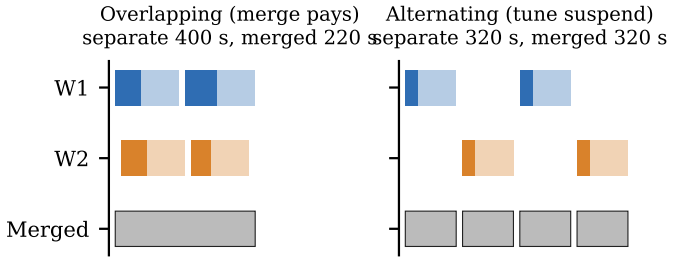


Fig. 1. Why overlap decides. Solid: queries; light: 60 s auto-suspend tails; grey: billed periods if merged. When two warehouses run at the same time, merging bills the shared seconds once; when they take turns, merging saves nothing that a short timeout has not already removed.

III. Related Work

Resource scaling inside the engine. Cloud warehouses increasingly decide how much compute to run on the customer's behalf. Amazon Redshift scales clusters with learned workload models [4]; ScaleSense estimates per-query resources to scale Alibaba AnalyticDB cost-effectively [5]; and the cost-intelligence vision proposes that users state budgets instead of sizes [6]. These systems operate inside one service on one workload. Our question is the customer's: how many warehouses to run, and which workloads to put together, under the billing contract as it stands.

Fleet traces. Snowset [7] and Redset [11] released query-level metadata from production fleets and have been used to characterise workloads—elasticity, repetition, the gap between benchmarks and real queries. To our knowledge neither has been used to evaluate customer-side layout decisions. Snowset's warehouse and database identifiers make that possible.

Database consolidation. Kairos [12] consolidates many database workloads onto fewer servers by predicting their combined resource use, and multi-tenant database services place tenants on servers to meet performance objectives at low cost [13]. Both minimise the hardware needed for a set of workloads, a capacity objective. Under per-second warehouse billing the customer pays for running time, not capacity, so the objective changes: a merge saves the seconds during which members would run concurrently (Proposition 1), and two workloads that fit easily on one server gain nothing if they never run at the same time.

VM consolidation. Server and virtual-machine consolidation packs workloads onto fewer hosts to save energy under capacity constraints [14]. There, the saving comes from capacity: fewer machines powered on. In a per-second-billed warehouse the saving comes from time—seconds that several warehouses would each have been billed for (Proposition 1)—so the question of *which* workloads to merge has a different answer, and our results show that idleness, the natural signal for VM consolidation, is the wrong one here.

Algorithm 1 Replay of one (merged) warehouse

Require: queries $(\text{arrival}, t, s_{\text{orig}})$; size s'; timeout a; slots $K = 8$; resume delay $r = 1\,\text{s}$

free $\leftarrow K$ slots free at $-\infty$; *up* $\leftarrow -\infty$ ▷ warehouse up until
for each query in order of arrival **do**
  start $\leftarrow \max(\text{arrival}, \text{earliest free slot})$
  if *start* $>$ *up* **then** *start* $\leftarrow$ *start* $+ r$ ▷ resume, billed
  end if
  end $\leftarrow$ *start* $+ t\,(s_{\text{orig}}/s')^{\alpha(c)}$; occupy the slot until *end*
  up $\leftarrow \max(up, end + a)$; latency $\leftarrow$ *end* $-$ arrival
end for
return billed seconds $\sum_C \max(60, |C|)$ over running periods; latencies

Practitioner guidance. Vendor guidance discusses auto-suspend settings and warehouse sizing [3]. We found no fleet-scale measurement of either remedy, or of consolidation, and no published rule for when to merge.

IV. Study Design

A. Data and groups

Snowset [7] records 69.2 million queries from 2,051 warehouses over 14 days in 2018. For each query it gives the warehouse id, warehouse size and database id, and the execution time; it does not identify the customer. As a same-customer proxy, a *candidate group* is two or more warehouses that each touch exactly one database, the same one, and keep one size for the whole trace. There are 134 such groups with 388 warehouses and 18.6 million queries. Most are small (89 pairs, 22 triples, 18 groups of four to six and 5 of seven or more), sizes range from X-Small to 4X-Large, and the groups are mostly light: the median group runs 3,345 queries in two weeks and would be billed 128 credits at default settings, while the largest runs 7.5 million queries. A query occupies its warehouse during $[\text{end} - \text{execution time},\ \text{end}]$.

B. Replay

The replay re-runs each warehouse's queries on a simulated warehouse of a chosen size and timeout (Algorithm 1). A query arrives at its observed execution start, which folds the original queueing into the arrival. Its execution time is rescaled to the replayed size by $t' = t\,(s/s')^{\alpha(c)}$, where c is a class by run time at the original size ($< 1\,\text{s}$, 1–10 s, 10–60 s, $> 60\,\text{s}$) and α is measured on Snowflake (Section V). Each warehouse has eight concurrent slots, Snowflake's default concurrency level, served first-come first-served. The first query after a suspend waits a one-second resume delay, which is billed. Billing follows Section II exactly.

C. Options and search

For each group we compare five options: (i) *as-is*: every warehouse at its size with AUTO_SUSPEND 600 s (Snowflake's default; we also report 60 and 300 s); (ii) *resize only*: each warehouse at half, the same or double its size; (iii) *tune auto-suspend only*: each warehouse at 60, 300 or 600 s; (iv) *consolidate all*: all members in one warehouse, at the best feasible size and timeout; (v) the *joint search* over partitions of the group into warehouses, a size for each (half, same or double any member's size) and a timeout (60, 300 or 600 s). Every option must keep each original warehouse's 95th-percentile latency within $1.2\times$ its as-is value. The joint search enumerates all set partitions for groups of up to four warehouses (Bell(4) $= 15$ partitions) and otherwise merges greedily, starting from separate warehouses and repeatedly applying the pairwise merge with the largest saving until none saves money; 13 of the 134 groups use the greedy path. "Best single knob" is the cheaper of options (ii) and (iii); the *extra saving* from consolidation is $1 - \text{joint}/\text{best single knob}$. Consolidation *pays* in a group when the joint plan merges warehouses, its extra saving exceeds 5%, and it saves at least one credit over the two weeks. The last two conditions exclude groups whose bill is so small that 5% is noise; the first excludes groups where the joint search wins only by changing size and timeout together, which neither single knob does (Section IV-E). Across all groups without a merge, that effect totals 0.5 credits, so the extra saving is consolidation's saving in all but name.

D. Features and rule

To explain when consolidation pays, we compute eleven group features from query history alone at 60 s auto-suspend: group size, spread of member sizes, total credits, and the mean and minimum over members of utilisation, resumes per day, idle share of billed time, and *overlap*—the share of a member's running periods that intersect any other member's running period. We report tie-corrected Spearman correlations with the extra saving and fit a depth-2 decision tree (minimum leaf size 5) for "pays", evaluated by leave-one-out cross-validation with balanced accuracy.

E. Pre-registration and deviations

The group definition, options, latency bound, replay parameters, size-model protocol, features, rule search and the seven sensitivity variants were written down with timestamps before the corresponding runs. Our stated expectation, written before any feature was computed, was that consolidation would pay "when members are lightly used, resume often, have high idle-tail share, low overlap, similar size". The data contradict it on overlap, idle share and size, and resume frequency does not matter (Section VI-C). The timestamps are the authors' own, recorded in the plan; they were not lodged with an external registry. The public repository was published after the main study, so its history starts then; the two additions after publication (a week-1/week-2 test of the rule and a real merge on Snowflake) were pushed to it before they were run. The plan was first written for the industry track under a working title; a dated addendum records the change to this track. We disclose nine deviations; the repository maps each to its dated entry in the plan. (1) The analysis keeps the 134 frozen groups although the runner also produced 21 groups after dropping size-changing members. (2) The 5% threshold for "pays" was fixed after the main run; Section VI-D shows the rule at 3% and 10%. (3) An independent review found that the

pre-registered analysis compared every option only on groups where *all* options were feasible, which dropped three groups (9, 17 and 2 warehouses) where consolidating everything breaks the latency bound—including the group with the largest consolidation saving. We compare as-is, single-knob and joint options on all 134 groups and report consolidate-all on the 131 where it is feasible; on the old 131-group set, auto-suspend saved 32%, the joint search 37%, and consolidation added 7.7%. (4) Our first rank correlation did not average tied ranks; we report tie-corrected values. (5) The SF100 size-scaling runs were added after the first block showed every original statement finishing in under 3 s at X-Small. (6) The pre-registered comparison of greedy against exhaustive search and the size model's held-out error were computed only after review, and the exhaustive check was then extended from groups of three or four warehouses to groups of five to eight by caching each block's best cost; all are reported in Section VI-D. (7) A second review found that the original definition of "pays" (extra saving above 5% alone) counted three groups that merge nothing and three that save under one credit; we tightened it as stated in Section IV. Under the original definition 27 groups pay and the rule's leave-one-out balanced accuracy is 0.76; every other conclusion is unchanged. (8) A third review asked whether the rule predicts the future; the week-1 to week-2 test (Section VI-C) was added and written down, with its expected outcome, before it was run, and pushed to the public repository with the plan. (9) The real merge on Snowflake (Section V-C) was added the same way; a session timeout cut three of its statements (reported there), and the analysis script's first event filter matched no events and was fixed before any metered number was read. Analyses marked *exploratory* were not pre-registered.

V. Calibration on Snowflake

A. Is the billing model right?

On a dedicated X-Small Generation 2 warehouse we issued short probe statements separated by gaps chosen around the timeout (20 to 600 s), at AUTO_SUSPEND 300 s and then 60 s, and compared the model's running periods, computed from the probes' execution intervals in `QUERY_HISTORY`, with the warehouse's own resume and suspend records in `WAREHOUSE_EVENTS_HISTORY`. We ran this check on a warehouse of its own, independently of any other experiment. Table I gives the result period by period. The model found every running period, and every modelled period is 0.3–1.3 s shorter than the actual one; the difference is the resume itself, which starts 0.2–1.0 s before the first query executes. Over each phase the model reproduces 99.8% (300 s) and 99.4% (60 s) of the running time, and the warehouse suspended a median 300.5 s and 60.1 s after its last query. The metered rate was 1.35 credits per running hour; the replay bills X-Small at one credit per hour, which scales every bill equally and leaves percentages unchanged.

One case is instructive. An administrator's session ran metadata statements (`GRANT`, `SHOW`) on the warehouse during the 60 s phase. Snowflake logs them in `QUERY_HISTORY` with the warehouse's name, but runs them in its cloud-services layer without keeping the warehouse up; counting them as warehouse work would overstate running time by 8%. A running-time model built from query history must exclude such statements.

TABLE I
Billing model vs. Snowflake's resume/suspend records, period by period (dedicated X-Small warehouse). Lengths in seconds; start relative to the phase's first resume.

AUTO_SUSPEND	start	probes	model	actual	model−actual
300 s	0	3	701.5	702.4	−1.0
	733	1	300.3	301.6	−1.3
	1335	3	703.5	704.2	−0.8
	2071	1	300.4	301.2	−0.8
60 s	0	3	125.8	126.1	−0.4
	142	1	60.3	60.7	−0.3
	263	1	60.5	60.8	−0.3
	504	3	126.2	127.1	−1.0
	647	1	60.6	61.0	−0.4
	768	1	60.6	61.1	−0.5
	1010	1	60.5	60.8	−0.3
Total, 300 s / 60 s (ratio)					0.998 / 0.994

B. How does run time scale with size?

On a second dedicated Generation 2 warehouse (query acceleration off) we ran 12 statements on our own TPC-H-style schema and 6 TPC-H statements on Snowflake's SF100 sample data at X-Small, Small and Medium, in ABBA order (XS, S, M, M, S, XS), three repetitions each with the first after every resize discarded, and took execution times from Snowflake's own records. For each statement and larger size, $\alpha = \log(t_{XS}/t_s)/\log(s)$; α for a class is the median over its statements (Table II). Sub-second statements barely speed up ($\alpha = 0.34$); 1–10 s statements scale sub-linearly (0.75); 10–60 s statements almost linearly (0.96). No statement exceeded 60 s, so the longest class borrows the 10–60 s value. Individual sub-second statements scatter widely, from $\alpha = -0.26$ (slower on a larger warehouse) to 1.02, which is why the sensitivity study varies α by $\pm 20\%$.

How good is the model? Predicting each of the 36 observations from its X-Small time and its class's α has a median error of 12.4% (90th percentile 44%). Held out by size, fitting α on Medium and predicting Small gives a median error of 12.4%, and fitting on Small and predicting Medium 24.8%. The two simple alternatives do much worse: assuming no speed-up errs by a median 60%, and assuming linear speed-up by 35%.

C. Does a real merge save what the replay predicts?

The checks above validate one warehouse at a time. The paper's subject is merging, so we also merged warehouses on Snowflake. From the trace we took 90-minute windows of three same-size warehouse pairs, chosen by a rule fixed in the plan before any run: the window with the highest predicted saving among groups where consolidation pays (*high overlap*), one in a different such group whose predicted saving is closest to 20% (*partial overlap*), and the window with the largest bills

TABLE II
SIZE SCALING MEASURED ON SNOWFLAKE: MEDIAN EXECUTION TIMES (S) AND PER-STATEMENT α FOR SMALL AND MEDIUM AGAINST X-SMALL; CLASS α IS THE MEDIAN WITHIN THE CLASS.

Statement	data	XS	S	M	α_S	α_M
Class < 1 *s: class* $\alpha = 0.34$						
point probe	TPCH10	0.05	0.04	0.04	0.34	0.23
part–supp cross	TPCH10	0.06	0.05	0.06	0.34	−0.07
top suppliers	TPCH10	0.15	0.13	0.21	0.13	−0.26
order priority	TPCH10	0.27	0.20	0.17	0.40	0.33
Q6 forecast	SF100	0.31	0.18	0.15	0.76	0.55
pricing summary	TPCH10	0.33	0.16	0.14	1.02	0.61
shipping priority	TPCH10	0.43	0.27	0.26	0.69	0.36
distinct parts	TPCH10	0.61	0.49	0.37	0.32	0.37
revenue by nation	TPCH10	0.69	0.49	0.46	0.49	0.29
part-type profit	TPCH10	0.69	0.67	0.64	0.04	0.05
Class 1–10 s: class $\alpha = 0.75$						
top customers	TPCH10	1.16	0.94	0.77	0.31	0.30
window rank	TPCH10	1.65	0.95	0.50	0.80	0.86
self join	TPCH10	2.28	1.36	0.79	0.74	0.77
Q3 shipping	SF100	2.54	1.63	1.26	0.64	0.51
Q1 pricing	SF100	4.93	2.54	1.37	0.96	0.92
distinct supp–part	SF100	6.13	4.09	2.03	0.58	0.80
Class 10–60 s: class $\alpha = 0.96$ *(also used for* > 60 *s)*						
Q18 large volume	SF100	13.60	6.97	3.67	0.97	0.95
top customers	SF100	20.67	13.12	5.04	0.66	1.02

among those predicted to save nothing (*turn-taking*). Each pair ran at X-Small with AUTO_SUSPEND 60 s on three dedicated warehouses at the same time—member A alone, member B alone, and both merged—with every statement replaced by `GENERATOR(TIMELIMIT` $\Rightarrow d$`)` of the trace duration d, sent at its trace offset (1,096 statements; median 0.1 s late). The replay's predictions were recorded and pushed to the public repository before the run.

Figure 2 shows Snowflake's metered credits. Merging the high-overlap pair saved 49.8% (predicted 49.8%), the partial-overlap pair 20.1% (predicted 20.0%), and the turn-taking pair 0.0% (predicted 0.0%): the saving is the overlap, as Proposition 1 says, and two warehouses that take turns gain nothing from sharing even though each is idle half the time. Run concurrently and merged, the billing model's running time on the executed statements matched Snowflake's resume and suspend records to within 0.31% on all nine warehouses, and the implied rate was the same on all nine (1.35–1.36 credits per running hour). One deviation: a session timeout of 300 s cut three turn-taking statements longer than that, equally on their own and on the merged warehouse, so that pair billed 75 rather than 83 minutes; the comparison is unaffected.

VI. RESULTS

A. RQ1: Where do the savings come from?

Figure 3 shows the billed compute each option saves relative to default settings across the 134 groups (127,018 credits over two weeks at the replay's rate). Tuning auto-suspend saves 23.7%; the joint search saves 35.0%. Per group, the median saving is 60% for tuned auto-suspend and 61% for the joint search, and Figure 4 shows that the two distributions nearly coincide: for most groups, the joint search's choice is simply a shorter timeout. In every one of the 253 warehouses or merged

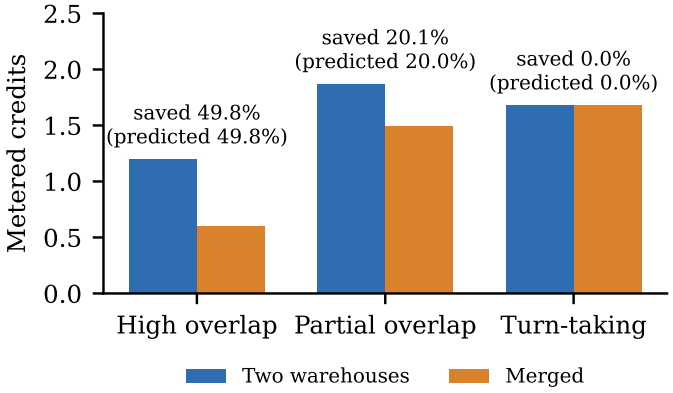


Fig. 2. A real merge on Snowflake: metered credits of two warehouses run separately and of the same statements on one merged warehouse (X-Small, AUTO_SUSPEND 60 s, 90 minutes each), with the saving the replay predicted before the run.

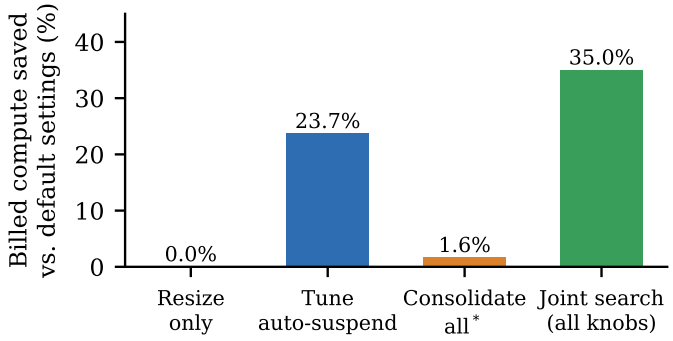


Fig. 3. Billed compute saved relative to default settings (AUTO_SUSPEND 600 s), 134 groups, two weeks. *Consolidate-all: the 131 groups where it meets the latency bound.

warehouses in the joint plans, the search picks the shortest timeout we allowed, 60 s.

What drives the auto-suspend saving? (exploratory) Across groups, the saving from tuning auto-suspend rises with the members' idle share of billed time (Spearman $+0.68$) and falls with the group's total credits (-0.75), its utilisation (-0.42) and its overlap (-0.57): the light, idle, turn-taking groups that make up most of the fleet are exactly the ones a short timeout helps most. Figure 5 shows a typical pair. Warehouse B runs short queries in regular bursts (19,450 queries over six days); warehouse A runs a few dozen queries a day (229 in all). At the 600 s default the two would run 51.0 hours over the trace, of which merging would save 1.0 hour (2%); at 60 s they run 21.8 hours separately and 21.7 merged. Tuning removes 57% of the bill; merging, almost nothing.

Resizing almost never pays. Resizing alone saves under 0.1% and is strictly cheaper in only three groups. This follows from the measured size model and the latency bound rather than being a separate discovery. Halving a warehouse slows sub-second queries by $2^{0.34} = 1.27\times$, past the $1.2\times$ bound.

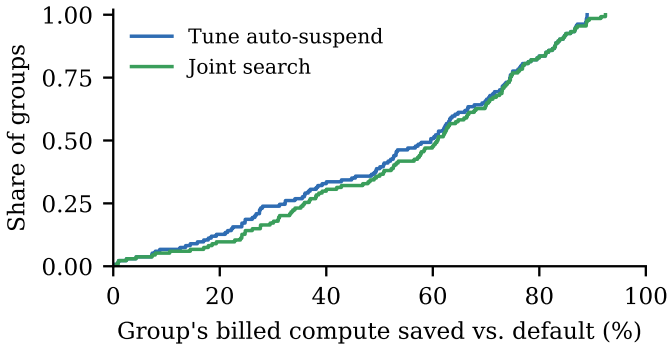


Fig. 4. Per-group saving relative to default settings, 134 groups. The two curves nearly coincide: for most groups the joint search adds little to tuning auto-suspend; the gap opens only in the few groups where consolidation pays.

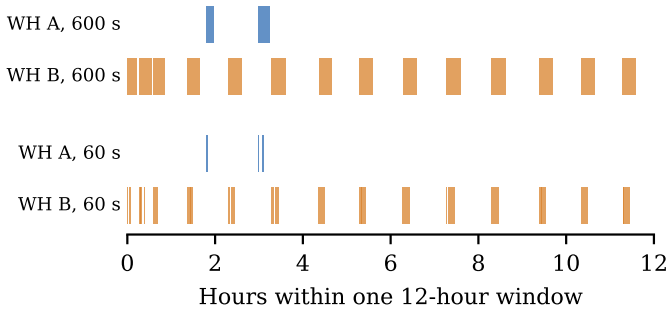


Fig. 5. A typical idle pair, twelve hours: running periods at the 600 s default (top) and at 60 s (bottom). The two warehouses take turns, so a short timeout removes most of the bill and merging removes almost none of what is left.

Doubling it costs twice the rate per second; with α near 1 for long queries the run is twice as fast, so the cost per query is unchanged, and with $\alpha = 0.34$ for short ones the run is only 21% faster, so the cost rises.

Merging everything is harmful. Consolidating every group into one warehouse, at its best size and timeout, is infeasible in three groups and saves only 1.6% in total on the other 131, although its per-group median is 52.5%: it costs more than tuning auto-suspend alone in 68 of 131 groups and more than the default settings in 19. A merged warehouse queues, and a group with members of different sizes must run at the largest rate.

What if timeouts are already short? Snowset records no suspend settings, so the as-is timeout is an assumption, and it decides how much tuning can save (Figure 6). Against an as-is timeout of 600 s the joint search saves 35.0% in total and a median 61% per group; against 300 s, 28.0% and 46%; against 60 s, 14.8% and 0%. With 60 s already in place, tuning has nothing left to remove—in no group was a longer timeout cheaper—and the remaining 14.8% is consolidation alone, concentrated in the same few groups. The consolidation result therefore does not depend on the assumption; the auto-suspend result does.

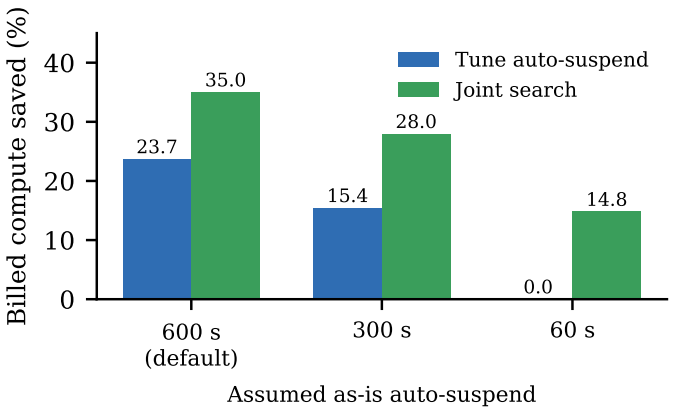


Fig. 6. Total saving of tuning auto-suspend and of the joint search against three assumed as-is timeouts (134 groups). With 60 s already in place, only consolidation is left to save.

TABLE III
SAVINGS BY GROUP SIZE. SUSPEND AND JOINT: % OF DEFAULT BILL; CONSOL.: EXTRA SAVING OF THE JOINT SEARCH AS % OF THE BEST SINGLE-KNOB BILL; PAYS: GROUPS WHERE CONSOLIDATION PAYS.

Warehouses	groups	wh.	suspend	joint	consol.	pays
2	89	178	35.6%	37.5%	3.0%	7
3	22	66	46.0%	49.4%	6.3%	5
4–6	18	85	24.4%	32.3%	10.4%	5
7+	5	59	6.4%	30.6%	25.9%	4
All	134	388	23.7%	35.0%	14.8%	21

By group size (exploratory). Table III breaks the savings down by the number of warehouses in a group. Tuning auto-suspend saves the most in pairs and triples and the least in the largest groups, whose warehouses are busier (median utilisation 32% for groups of seven or more, against 3% for pairs). Consolidation grows with group size: it adds 3% of the tuned bill for pairs, 6% for triples, 10% for groups of four to six and 26% for the five groups of seven or more.

B. RQ2: What does consolidation add?

Relative to the best single knob, the joint search adds a median of 0% (interquartile range 0–2.2%) and 14.8% of the total. It merges warehouses in 83 groups, but consolidation pays in only 21; where it does, it cuts the group's best single-knob bill by 5.1–70.3% (median 12.3%). It forms 91 merged warehouses, 69 of them pairs. Table IV lists all 21 groups.

How much rests on one group? Much of it. One group accounts for 68.5% of the consolidation saving and another for 17.5%. Without the 17-warehouse group, auto-suspend saves 25.3%, the joint search 29.4%, and consolidation adds 5.5% of the tuned total instead of 14.8%, in 20 groups; without the two largest, 2.8%. The median-group result does not change. Consolidation is a large win for a few organisations and a small one for the fleet.

Case study: the 17-warehouse group (exploratory). Figure 7 shows six hours of this group's running periods at 60 s.

TABLE IV

ALL 21 GROUPS WHERE CONSOLIDATION PAYS (A MERGE, EXTRA SAVING $> 5\%$ AND AT LEAST ONE CREDIT SAVED), ORDERED BY CREDITS SAVED. SIZES IN CREDITS PER HOUR; BEST: CHEAPER OF RESIZE-ONLY AND SUSPEND-ONLY; JOINT: JOINT SEARCH (CREDITS, TWO WEEKS); SHARE: OF THE FLEET'S TOTAL CONSOLIDATION SAVING; OVERLAP: MEAN OVERLAP OF MEMBERS' RUNNING PERIODS; FLAGGED: OVERLAP > 0.617, THE RULE OF SECTION VI-C (OVERLAP SHOWN TO TWO DECIMALS; ROW 7 IS 0.621); MERGED: NUMBER OF MEMBERS IN EACH MERGED WAREHOUSE OF THE JOINT PLAN.

#	k	sizes	queries	best	joint	extra	share	overlap	flagged	merged
1	17	1–8	3,712,733	13,995	4,156	70.3%	68.5%	1.00	✓	3 + 6 + 8
2	6	4,8,8,16,32,64	329,968	10,994	8,476	22.9%	17.5%	1.00	✓	4
3	6	2,4,4,8,8,8	181,535	2,992	2,557	14.5%	3.0%	1.00	✓	6
4	3	1,1,2	632,030	948	638	32.7%	2.2%	1.00	✓	3
5	4	8,8,16,32	48,930	3,916	3,690	5.8%	1.6%	0.74	✓	2
6	2	2 (all)	238,553	832	640	23.1%	1.3%	1.00	✓	2
7	3	4,4,8	16,628	488	363	25.7%	0.9%	0.62	✓	2
8	5	1,1,1,4,4	1,225,708	1,075	975	9.3%	0.7%	0.89	✓	2 + 2
9	2	2,8	8,931	1,084	1,016	6.3%	0.5%	0.75	✓	2
10	18	32 (all)	1,012	222	161	27.5%	0.4%	0.67	✓	2 + 2 + 4 + 3 + 2 + 3
11	7	1–8	77,094	442	386	12.7%	0.4%	0.96	✓	6
12	3	1 (all)	11,460	85.9	52.4	39.0%	0.2%	0.67	✓	3
13	2	1 (all)	376,546	228	196	13.9%	0.2%	0.48		2
14	2	4 (all)	20,985	386	358	7.3%	0.2%	0.36		2
15	3	4,8,8	5,335	249	225	9.9%	0.2%	0.81	✓	2
16	2	4 (all)	11,246	148	130	12.3%	0.1%	0.33		2
17	8	1–4	32,582	181	164	9.3%	0.1%	0.68	✓	4
18	2	1 (all)	41,773	79.7	75.2	5.6%	0.0%	0.29		2
19	2	1 (all)	3,321	32.0	29.4	8.2%	0.0%	0.34		2
20	4	1,4,4,4	7,179	50.5	47.9	5.1%	0.0%	0.34		2
21	3	1 (all)	7,295	27.3	25.2	7.5%	0.0%	0.66	✓	3

Over the 13.3 days of the trace, at least four of its warehouses were running at every moment, and 12.8 on average. Run separately, the 17 warehouses accumulate 4,093 warehouse-hours of running time; billed at their sizes this is the 13,995 credits of the single-knob option (13,991 before the billed one-second resume delays). The search merges them into three warehouses (sizes 1, 4 and 8, all at 60 s), which each run for the whole trace, 960 warehouse-hours in all, and cost 4,156 credits (70.3% less). Merging does not reduce how long compute runs—the merged warehouses never suspend—but how many warehouses are billed for the same seconds, exactly as Proposition 1 predicts. The merged warehouses stay small, no larger than the largest member, and still meet the latency bound. This is the case our expectation got backwards: the members are busy, not idle—mean utilisation 42%, against a median of 4% across groups—and they are worth merging precisely because they are running at the same time.

Could they be one customer's multi-cluster warehouse recorded as many? Not as recorded: Snowset gives each a separate warehouse id, and the members have four different sizes (1, 2, 4 and 8), while the clusters of one multi-cluster warehouse share a size. The search's merged warehouses also mix sizes. A multi-cluster warehouse is, however, a natural way to realise such a merge in practice, and a platform team should weigh it.

Two smaller groups show the same pattern at other scales. In a six-warehouse group (sizes 4, 8, 8, 16, 32 and 64) the search merges the four smallest into one size-16 warehouse, keeps the 32 and 64 separate, and cuts 10,994 to 8,476 credits (22.9%). In a three-warehouse group (sizes 2, 1, 1; 632,030 queries; at least 98.9% of each member's running periods overlap another's) merging all three into one size-2 warehouse cuts 948 to 638 credits (32.7%).

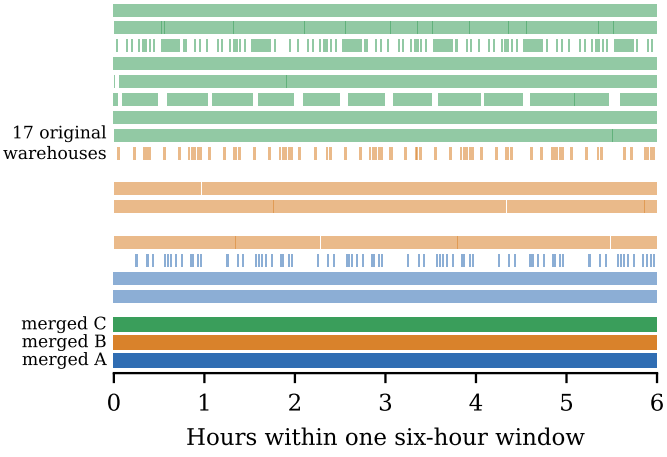


Fig. 7. Case study: six hours of the 17-warehouse group at 60 s auto-suspend. Upper rows: each original warehouse's running periods, coloured by the merged warehouse the search assigns it to. Lower rows: the three merged warehouses, which run continuously. Merging cuts the group's two-week bill from 13,995 to 4,156 credits.

C. RQ3: When does consolidation pay?

Among the eleven features (Table V), the mean overlap of members' running periods correlates most with the extra saving (Spearman $+0.51$), as Proposition 1 predicts. Group size follows ($+0.32$); a wide spread of member sizes is negative (-0.24), since merging a small warehouse into a large one bills its queries at the large rate. Idle share is weakly *negative* (-0.20), utilisation weakly positive ($+0.13$), and resume frequency near zero: the idle, lightly used groups we expected to gain are the ones that gain least. Figure 8

TABLE V

SPEARMAN CORRELATION OF GROUP FEATURES WITH THE EXTRA SAVING FROM CONSOLIDATION (134 GROUPS; TIE-CORRECTED). ALL FEATURES ARE COMPUTED FROM QUERY HISTORY AT 60 S AUTO-SUSPEND.

Feature	ρ	Feature	ρ
Overlap, mean	+0.51	Idle share, min	−0.22
Overlap, min	+0.40	Idle share, mean	−0.20
Group size	+0.32	Resumes/day, mean	−0.06
Size spread	−0.24	Resumes/day, min	−0.07
Credits (60 s)	+0.20	Utilisation, mean	+0.13
Utilisation, min	+0.12		

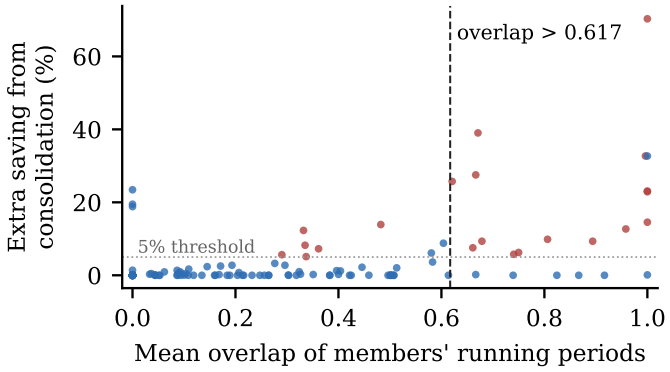


Fig. 8. Extra saving from consolidation over the best single-knob option vs. the mean overlap of members' running periods (134 groups). Red: the 21 groups where consolidation pays (a merge, extra saving $> 5\%$, at least one credit). Dashed: the rule's threshold, 0.617.

shows the pattern.

Does the billing argument explain the savings? (exploratory) This closes the argument begun in Figure 1. Proposition 1 predicts the saving of a merge from coverage alone: the overlap of the members' running periods, valued at the merged warehouse's rate, minus the cost of moving members of other sizes to that rate. It ignores queueing, size scaling and resume delays, which the replay includes. For each of the 91 merged warehouses in the joint plans we computed both (Figure 9). For the 79 whose members share one size and keep it, the replay realised 1,342 credits of saving against 1,340 predicted (median ratio per merge 1.003; Spearman 0.99). For the 12 mixed-size merges, the prediction is 12,859 credits against 12,998 realised; the size change costs 12,692 credits, about half of the overlap these merges recover. In two mixed-size merges the replay realises more than predicted, because the small member's queries run faster on the larger warehouse and shorten its coverage. In this fleet, then, consolidation's saving is the overlap the billing rule double-counts, net of the rate change; with eight concurrency slots, queueing plays almost no part (with four it does; Section VI-D).

A rule. A depth-2 decision tree, evaluated by leave-one-out cross-validation, splits twice on the same feature and so reduces to one rule—*consolidate when mean overlap exceeds about 0.6* (0.617 when fitted on all groups)—with balanced

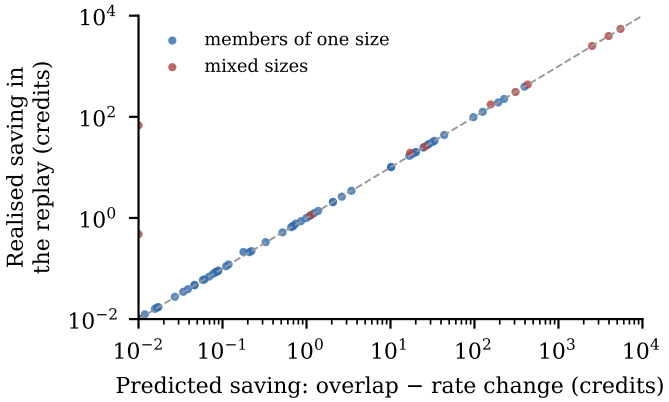


Fig. 9. Saving of each merged warehouse in the joint plans: realised in the replay vs. predicted from overlap and rate change alone (Proposition 1). Merges with no saving sit at the lower left. Dashed: equality.

accuracy 0.80, precision 0.64 and recall 0.67 (baseline 0.50). A single threshold, also chosen inside each fold, lands lower (0.33) and reaches 0.82, but with precision 0.36: overlap above one third is necessary rather than sufficient.

Does the rule predict the next week? A platform team would look at past history, decide, and live with the result. We therefore fitted the rule on the first week of the trace and applied it to the second, as pre-registered after review (deviation (8)). The 72 groups whose members all run in both weeks hold 96% of the consolidation saving. Fitted on week 1, the threshold lands lower (overlap > 0.35); applied to week-1 features, it predicts which groups pay in week 2 with balanced accuracy 0.88 (precision 0.52, recall 1.00), and the full-trace threshold of 0.617 used the same way reaches 0.86 (precision 0.75, recall 0.80). Acting on history—each flagged group adopts its week-1 plan unchanged—captures 6,614 of the 6,637 credits that the best week-2 plans save by consolidation. Week-1 latency targets are missed in week 2 by 35 groups, but 36 miss them with no change at all: p95 drifts between weeks, and the plans do not add to it.

Where the money is, and what the rule misses (exploratory). The eight groups with the largest consolidation saving hold 96% of it; five of them have a mean overlap of 1.0 and seven are above 0.617. Scored against the 21 groups where consolidation pays, the rule flags 15 correctly; those 15 hold 97.5% of the fleet's consolidation saving. It misses 6, mostly pairs of equal-size warehouses with low overlap (median 0.34) that gain a median 8% but together account for only 0.6% of the saving. It wrongly flags 7, which gain a median of 0%; their members differ in size by a median factor of four, so merging moves small warehouses' queries onto a large warehouse's rate—the second cost noted after Proposition 1. Size spread is a cost, not a veto: the largest mixed-size merges are also the largest wins (Figure 9). The rule is therefore a "look here first" filter: it finds almost all of the money, and its false alarms are mixed-size groups whose overlap does not outweigh the rate change, which a replay

TABLE VI

SENSITIVITY (134 GROUPS EACH): SAVINGS VS. DEFAULTS; CONSOL.: EXTRA SAVING AS % OF THE TUNED TOTAL; PAYS: GROUPS WHERE CONSOLIDATION PAYS; RULE: IN-SAMPLE BALANCED ACCURACY OF THE RULE FIXED BEFORE THESE RUNS (MEAN OVERLAP > 0.55).

Setting	suspend	joint	consol.	pays	rule
Main setting[†]	23.7%	35.0%	14.8%	21	0.81
Latency bound 1.1×	23.6%	34.2%	13.9%	21	0.81
Latency bound 1.5×	23.7%	35.5%	15.4%	20	0.86
4 concurrency slots	23.0%	27.0%	5.3%	11	0.74
Resume delay 0 s	23.7%	35.0%	14.8%	19	0.85
Resume delay 5 s	22.3%	34.4%	15.5%	24	0.79
$\alpha \times 0.8$	23.7%	35.0%	14.8%	20	0.83
$\alpha \times 1.2$[‡]	23.7%	35.3%	15.2%	21	0.81

[†] 1.2× bound, 8 slots, 1 s resume delay, measured α.

[‡] Gives $\alpha > 1$ for queries over 10 s; an optimistic bound.

before acting rejects.

D. RQ4: How robust are the answers?

Sensitivity. Table VI repeats the full replay under seven one-at-a-time changes, each on all 134 groups. In every setting auto-suspend saves 22–24%, the median extra from consolidation is at most 0.1%, and overlap is the strongest feature (Spearman $+0.42$ to $+0.59$). The rule fixed before the sensitivity runs (overlap > 0.55, from the original analysis) keeps a balanced accuracy of 0.74–0.86. Concurrency matters most: with four slots instead of eight, merged warehouses queue, consolidation pays in only 11 groups and adds 5.3% of the tuned total instead of 14.8%; the six-warehouse group above falls from 22.9% to 0.8%, while the 17-warehouse group keeps 27.6%. A larger latency allowance and a zero resume delay change little; a five-second resume delay makes every resume costlier and so makes merging pay in more groups (24).

Threshold. At 3% and 10% instead of 5% (with the same other conditions), 22 and 11 groups pay, and the rule's balanced accuracy is 0.82 and 0.84 (precision 0.62 and 0.35).

Greedy against exhaustive search. The joint search is exhaustive for groups of up to four warehouses and greedy beyond. To measure what greedy loses, we re-ran all 32 groups of three or four warehouses with the greedy search as well (for pairs the two coincide), and ran exhaustive search on the 10 greedy groups of five to eight warehouses (up to Bell(8) $= 4{,}140$ partitions, with each block's best size and timeout cached). Greedy found the same cost as exhaustive search in all 42 groups, and every re-run matched the main run exactly. Only the groups of 9, 17 and 18 warehouses remain unchecked (Bell(17) $\approx 8 \times 10^{10}$ partitions), so we cannot rule out a better plan there; their reported consolidation savings are lower bounds.

Size model. The size model's held-out error (Section V) is 12–25%, against 35–60% for the simple alternatives, and the $\pm 20\%$ variants of α leave every conclusion unchanged.

Cost of the study. The replay and search took about two hours in total on a laptop (median 0.3 s per group; the largest group 57 minutes), so a platform team can run the same analysis on its own fleet.

```
WITH tail AS (SELECT wh, s, e + 60 AS e FROM q),
reach AS (SELECT wh, s, e, MAX(e) OVER
    (PARTITION BY wh ORDER BY s, e ROWS BETWEEN
     UNBOUNDED PRECEDING AND CURRENT ROW) AS hi FROM tail),
gap AS (SELECT wh, s, e, CASE WHEN s > LAG(hi) OVER
    (PARTITION BY wh ORDER BY s, e) THEN 1 ELSE 0
    END AS brk FROM reach),
run_id AS (SELECT wh, s, e, SUM(brk) OVER (PARTITION
    BY wh ORDER BY s, e ROWS UNBOUNDED PRECEDING) AS r
    FROM gap),
period AS (SELECT wh, r, MIN(s) AS a, MAX(e) AS b
    FROM run_id GROUP BY wh, r),
shared AS (SELECT p.wh, p.r, MAX(CASE WHEN o.wh IS NULL
    THEN 0 ELSE 1 END) AS h FROM period p LEFT JOIN
    period o ON o.wh <> p.wh AND o.a <= p.b AND o.b >= p.a
    GROUP BY p.wh, p.r)
SELECT AVG(ov) AS mean_overlap FROM
  (SELECT wh, AVG(h) AS ov FROM shared GROUP BY wh) per_wh;
```

Fig. 10. Mean overlap of a candidate group's running periods at 60 s auto-suspend. Input: `q(wh, s, e)`, the execution intervals of the group's queries. Consolidate when the result exceeds about 0.6.

VII. LESSONS FOR PLATFORM TEAMS

The results suggest an order of operations. *First*, set auto-suspend per warehouse from its own history; in our data this captured two thirds of the available saving and needs no organisational change. *Second*, for warehouses that serve the same workload owner, compute the overlap of their running periods. Figure 10 computes it from query history with a 60 s timeout using only window functions; on the seven Snowset groups we re-ran, it reproduced the paper's overlap feature exactly, and it took 4 s in DuckDB for the 17-warehouse group's 3.7 million queries. On Snowflake the input can come from `QUERY_HISTORY`—excluding statements that run only in the cloud-services layer—or the periods can be read directly from `WAREHOUSE_EVENTS_HISTORY`. *Third*, consider merging only groups whose mean overlap exceeds about 0.6 and replay the merge against a latency bound before acting. Size spread is a cost, since merged queries run at the largest member's rate, but high overlap can outweigh it—most of the money in our fleet came from mixed-size merges—so replay such merges rather than rejecting them. Merged warehouses also queue, and with low concurrency limits the benefit shrinks. *Do not* merge warehouses merely because they are idle most of the time; if they take turns, a shorter timeout already removes their tails.

VIII. DISCUSSION

Why the intuition fails. The case for consolidation is usually made in terms of idle time: many warehouses each pay for their own idle tails, so put them together and pay once. Under per-second billing with an idle timeout that is only half right. The tails are paid once only if they coincide; if the warehouses take turns, the merged warehouse pays for each member's tail in turn, exactly as before. What the billing rule actually charges twice is concurrent running time, so the signal is concurrency, and idleness points the wrong way: idle warehouses rarely run at the same time. Shortening the timeout removes most of the idle time directly, without any change to who shares what, which is why it dominates.

What this means for other platforms. The argument depends only on the shape of the contract: a resource billed by the second while it runs, kept up by a timeout after the last request. Databricks SQL warehouses [8] and the minimum-hold periods of BigQuery autoscaling [10] have that shape; Redshift Serverless bills per second with a minimum but no idle tail [9], so there the overlap term is all there is to gain. The numbers in this paper are specific to Snowset and to our replay; the direction of the effect, and the overlap test, should carry over.

What a vendor could do. The largest saving we found came from a customer who ran many busy warehouses side by side. Such a customer is paying for the same seconds many times, and the platform has every input needed to notice: running periods per warehouse are already recorded in the events history. Surfacing the overlap of a customer's warehouses, alongside the existing advice on auto-suspend, would point platform teams at the few merges worth making. Multi-cluster warehouses, which add clusters under load and remove them when idle, already implement a form of this merge for workloads that share a size.

IX. Threats to Validity

External validity. Snowset is from 2018 and ran on older hardware; workloads and engines have changed. Our size model was measured on 2026 Generation 2 warehouses on 18 statements, none longer than 60 s, so scaling for longer queries is extrapolated. Groups are a database-level proxy for one customer, because Snowset has no customer id; some groups may span customers, and some customers' related warehouses touch several databases and are excluded. The fleet is one vendor's; the billing argument (Proposition 1) carries over to services with the same contract shape, the numbers need not.

Internal validity. Snowset records no suspend settings, so the as-is timeout is assumed; we report 60, 300 and 600 s. Arrivals are observed execution starts, so original queueing is folded in. Concurrency is modelled as eight first-come first-served slots, not Snowflake's resource-based limit; four slots change the size of the consolidation result but not its pattern. Result-cache and local-cache effects are not modelled; merging could warm or cool caches. The timeout grid stops at 60 s.

Construct validity. The billing validation used one X-Small warehouse with sequential probes; the merge test (Section V-C) adds concurrency and merging, but on three pairs, at X-Small, with synthetic statements of the trace's durations rather than the original queries, and without size changes. The 60-second minimum cannot bind in any option we evaluate: every option uses AUTO_SUSPEND $\geq$ 60 s, so every running period lasts at least 60 s (Section II). The replay still applies the minimum, and the unit tests check it on 200 random workloads, binding below 60 s and never at or above (`tests/test_advisor.py`). The "pays" threshold was set after the main run (we report 3% and 10%). The rule rests on 134 groups with 21 positives, and most of the saving sits in two groups; it should be read as a filter for where to look, validated by a replay, not as a classifier to act on alone.

X. Artifacts

The code, the raw result files, the Snowflake calibration logs and Snowflake's own records for the calibration warehouses, and the pre-registration with every deviation are public at https://github.com/khadarvalishaik0520/warehouse-consolidation-study. One command regenerates every number, table and figure in this paper in minutes from the result files and two small extracts of Snowset shipped with the repository; the replay itself is re-run from the full trace with two further commands.

XI. Conclusion

For a fleet of cloud warehouses, the first and largest saving is the auto-suspend timeout. Resizing rarely helps under a latency bound. Consolidation is not a general remedy: it adds nothing for the median group, and merging everything is often worse than doing nothing. It pays—in one group by more than two thirds of the tuned bill—when warehouses run at the same time, because the billing rule charges every running warehouse for the same second; query history is enough to find those groups.

Acknowledgment

AI-generated content: Claude (Anthropic) was used to draft and edit the text of all sections, to write the replay, search and analysis code behind Sections IV–VI and the SQL in Figure 10, and to produce all figures and tables. The authors directed the study and the first author ran the experiments; every reported number was re-computed from the result files, and the authors take full responsibility for the content.

References

[1] Snowflake Inc., "Overview of warehouses," https://docs.snowflake.com/en/user-guide/warehouses-overview, accessed Oct. 2026.

[2] ——, "CREATE WAREHOUSE," https://docs.snowflake.com/en/sql-reference/sql/create-warehouse, accessed Oct. 2026.

[3] ——, "Warehouse considerations," https://docs.snowflake.com/en/user-guide/warehouses-considerations, accessed Oct. 2026.

[4] V. Nathan, V. Singh, Z. Liu, M. Rahman, A. Kipf, D. Horn, D. Pagano, G. Saxena, B. Narayanaswamy, and T. Kraska, "Intelligent scaling in Amazon Redshift," in *Companion of the 2024 International Conference on Management of Data (SIGMOD-Companion)*, 2024.

[5] Y. Wu, Y. Li, Z. Wang, K. Chen, L. Shou, Z. Chen, L. Lin, H. Li, and G. Chen, "ScaleSense: Cost-intelligent scaling framework via learned resource estimation in Alibaba AnalyticDB," *Proc. VLDB Endowment*, vol. 19, no. 12, pp. 4063–4075, 2026.

[6] H. Zhang, Y. Liu, and J. Yan, "Cost-intelligent data analytics in the cloud," in *Conference on Innovative Data Systems Research (CIDR)*, 2024.

[7] M. Vuppalapati, J. Miron, R. Agarwal, D. Truong, A. Motivala, and T. Cruanes, "Building an elastic query engine on disaggregated storage," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 449–462.

[8] Databricks, "Create a SQL warehouse," https://docs.databricks.com/sql/admin/create-sql-warehouse.html, accessed Oct. 2026.

[9] Amazon Web Services, "Billing for on-demand compute capacity (Amazon Redshift Serverless)," https://docs.aws.amazon.com/redshift/latest/mgmt/serverless-billing-on-demand.html, accessed Oct. 2026.

[10] Google Cloud, "Introduction to slots autoscaling (BigQuery)," https://cloud.google.com/bigquery/docs/slots-autoscaling-intro, accessed Oct. 2026.

[11] A. van Renen *et al.*, "Why TPC is not enough: An analysis of the Amazon Redshift fleet," *Proc. VLDB Endowment*, vol. 17, no. 11, pp. 3694–3706, 2024.

[12] C. Curino, E. P. C. Jones, S. Madden, and H. Balakrishnan, "Workload-aware database monitoring and consolidation," in *Proc. ACM SIGMOD International Conference on Management of Data*, 2011, pp. 313–324.

[13] W. Lang, S. Shankar, J. M. Patel, and A. Kalhan, "Towards multi-tenant performance SLOs," *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 6, pp. 1447–1463, 2014.

[14] A. Beloglazov and R. Buyya, "Optimal online deterministic algorithms and adaptive heuristics for energy and performance efficient dynamic consolidation of virtual machines in cloud data centers," *Concurrency and Computation: Practice and Experience*, vol. 24, pp. 1397–1420, 2012.